Workiz Integration

1. Overview

1.1 Description

Workiz Integration connects **Workiz** (field service management platform) with **Newo Platform**.

It enables:

- Creating service jobs in Workiz with full customer details, address, job type, and scheduling
- Automatic technician assignment to created jobs
- Cancelling existing jobs by setting status to "Canceled"
- Looking up existing jobs for returning customers by phone number
- Syncing dispatcher team members from Workiz for technician assignment
- DevMode for testing without real API credentials (LLM-emulated responses)
- Auto-configuring conversation canvas with booking and cancellation scenarios

This integration is designed for **home services businesses** (appliance repair, plumbing, electrical, HVAC, cleaning, etc.) who use Workiz to manage their field operations and need **an AI receptionist to book and cancel jobs** through voice or chat.

1.2 Use Cases

- When a caller requests a service → Agent collects details (name, phone, email, address, job type, date/time) and creates a job in Workiz
- When a caller wants to cancel an appointment → Agent finds the booking and cancels it in Workiz
- When a returning customer calls → Agent looks up their existing jobs by phone number
- When the project is published → Agent syncs the dispatcher team list from Workiz for technician assignment

1.3 Supported Features

Feature	Supported	Notes
Create Job	✅	Full customer details, address, job type, date/time
Cancel Job	✅	Sets job status to "Canceled" with optional notes
Technician Assignment	✅	Auto-assigns primary tech from dispatcher team
Existing Client Lookup	✅	Search jobs by phone number, filters future bookings
Team Sync	✅	Loads dispatcher-capable team members on publish
Canvas Auto-Setup	✅	Booking + cancellation scenarios and intents
DevMode (LLM Emulation)	✅	Test without real API — LLM generates realistic responses
Check Availability	❌	Workiz API does not support availability queries
Reschedule	❌	Cancel + rebook as workaround
Payment Processing	❌	Not implemented

2. Requirements

Before installation:

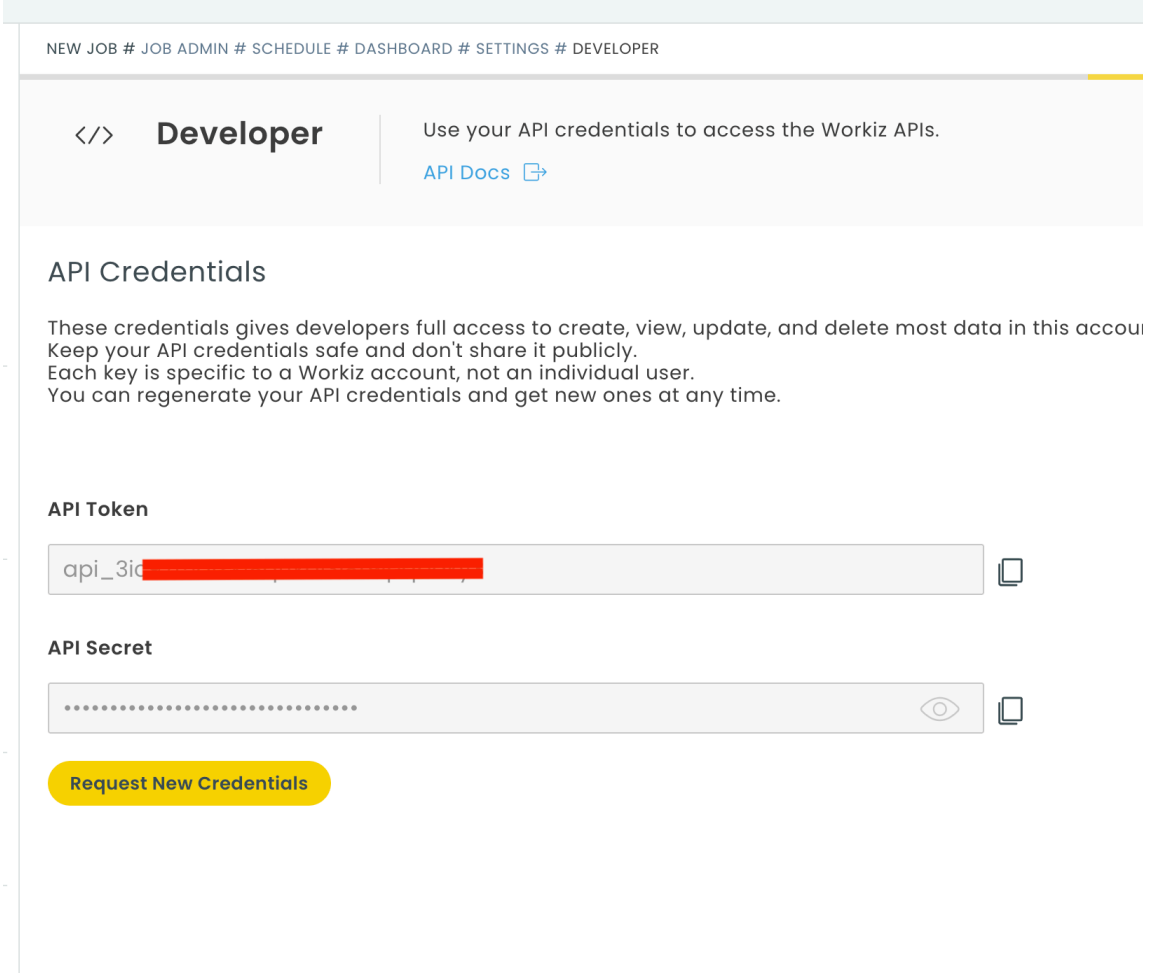
- Active **Workiz** account with API access
- **API Key** from Workiz Settings → Integrations → API
- **API Secret** from the same page
- At least one team member with a dispatcher-capable role (dispatch, manager, or admin)

3. How to Setup

3.1 Step 1 — Get API Credentials from Workiz

1. Log in to your **Workiz** account at https://app.workiz.com
2. Navigate to **Settings** → **Integrations** → **API**
3. Copy the **API Token** (this is your API Key)
4. Copy the **API Secret**
5. Note your **Job Types** from **Settings** → **Job Types** (e.g., "Dishwasher", "Dryer", "Refrigerator")

> 📸 *Screenshot: Workiz Settings → Integrations → API page showing API Token and Secret*

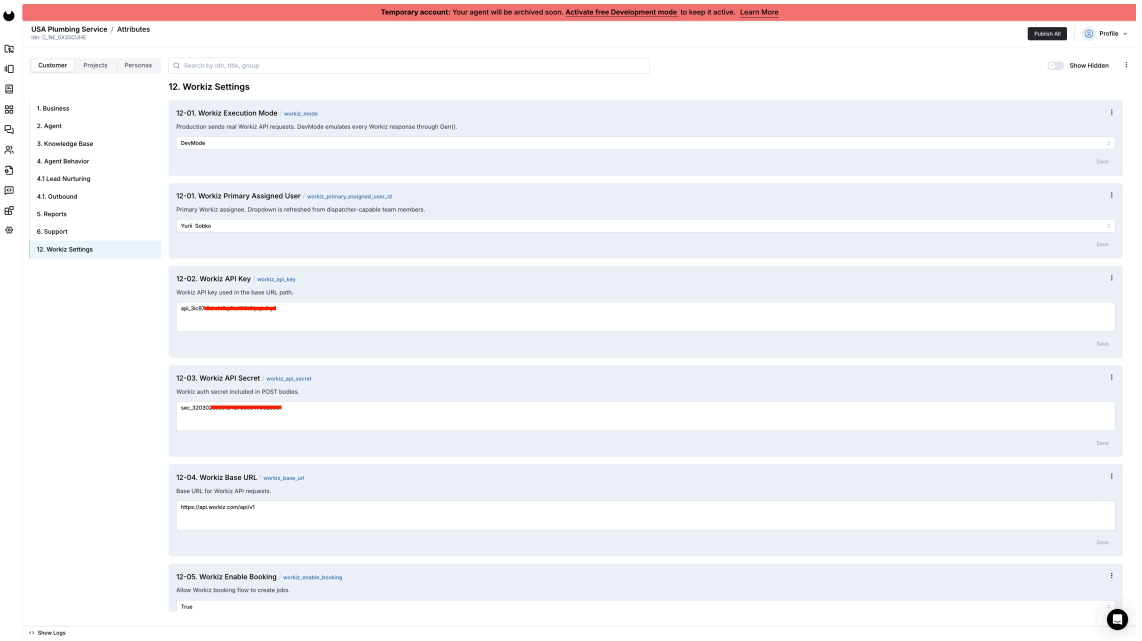


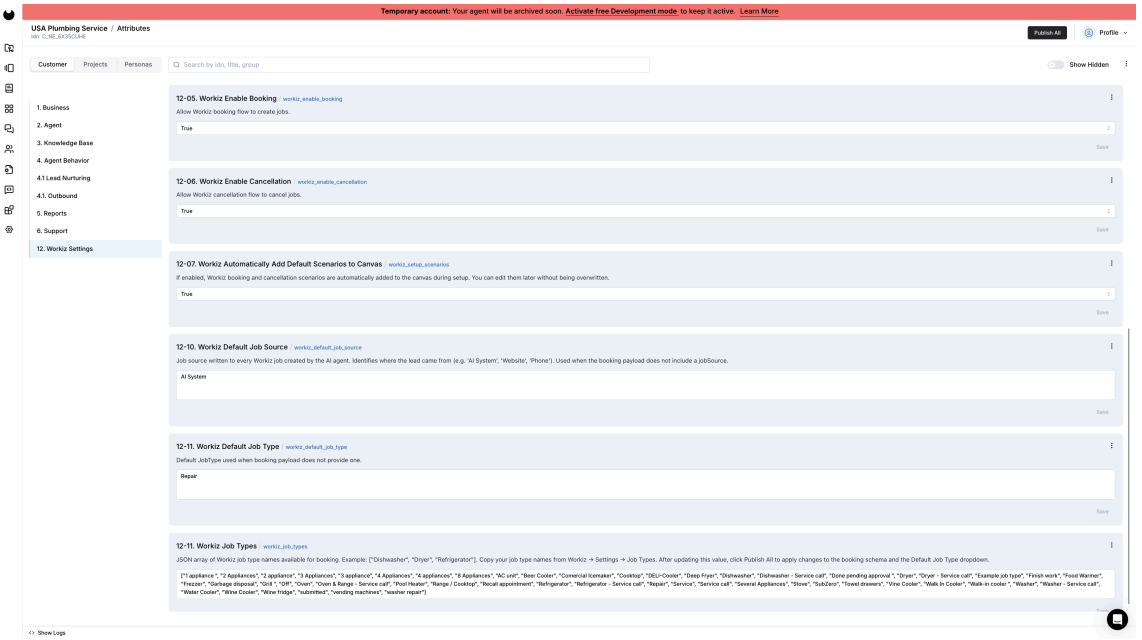
3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Default	Description
`workiz_api_key`	✅	—	API Token from Workiz Settings
`workiz_api_secret`	✅	—	API Secret from Workiz Settings
`workiz_base_url`	❌	`https://api.workiz.com/api/v1`	API base URL
`workiz_mode`	❌	`Production`	`Production` for real API, `DevMode` for LLM emulation
`workiz_job_types`	❌	`[]`	JSON array of job type names, e.g. `["Dishwasher", "Dryer"]`
`workiz_enable_booking`	❌	`True`	Enable job creation
`workiz_enable_cancellation`	❌	`True`	Enable job cancellation
`workiz_setup_scenarios`	❌	`True`	Auto-add booking/cancellation scenarios to canvas
`workiz_primary_assigned_user_id`	❌	—	Primary technician (dropdown populated after team sync)
`workiz_default_job_source`	❌	`AI System`	Job source label in Workiz

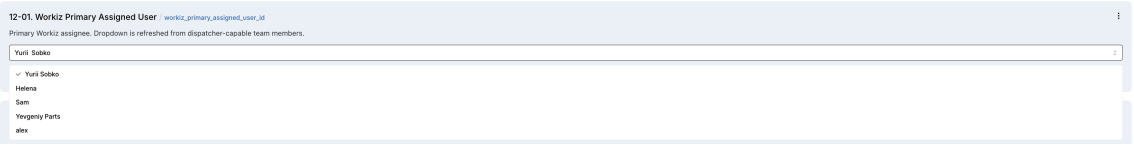
> 📸 *Screenshot: Newo Platform → Builder → Attributes showing Workiz settings*





3. Click **Save + Publish All**
4. On publish, the InitFlow automatically:
    - Creates HTTP connector for Workiz API
    - Sets all customer attributes with metadata
    - Registers booking and cancellation tools on ConvoAgent
    - Syncs dispatcher team members from Workiz
    - Populates the technician assignment dropdown

> 📸 *Screenshot: Workiz Primary Assigned User dropdown populated after publish*



4. How to Use

4.1 How the Integration Works

The integration uses Workiz **Jobs API** for creating and cancelling jobs, and **Team API** for syncing dispatcher members. Authentication uses API Key (in URL path) and API Secret (in request body).

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Workiz

Available Triggers

Trigger	Event	Description
Create Job	`book_slot`	When the agent confirms a booking
Cancel Job	`convo_agent_cancel_booking`	When the agent processes a cancellation
Existing Client Lookup	`define_user_phone_number`	When identifying a returning caller
Setup	`project_publish_finish`	Initialize integration on deploy
Canvas Setup	`gm_workflow_builder_canvas_built`	Auto-add scenarios to canvas

Available Actions

- **Create Job** — Submit new service job (POST `/job/create/`)
- **Assign Technician** — Assign primary tech to job (POST `/job/assign/`)
- **Cancel Job** — Set job status to Canceled (POST `/job/update/`)
- **Get Team** — Fetch dispatcher team members (GET `/team/all/`)
- **Get Jobs** — Fetch existing jobs for client lookup (GET `/job/all/`)

4.2 Flows

Flow	Purpose
InitFlow	Initialize integration: connectors, attributes, team sync
NetworkFlow	HTTP execution layer with Production/DevMode routing
ApiFlow	All domain API calls: create job, cancel, assign tech, get team
BookingFlow	Entry point for booking — validates and delegates to ApiFlow
CancellationFlow	Entry point for cancellation — validates and delegates to ApiFlow
ExistingClientFlow	Lookup existing jobs by phone number

CanvasSetupFlow	Auto-add booking + cancellation scenarios to canvas
LogFlow	Centralized debug logging

4.3 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Workiz Integration
    participant API as Workiz API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /team/all/
    API-->>I: Team members
    I->>I: Filter dispatchers, populate dropdown

    Note over U,API: Booking
    U->>A: "I need my dishwasher repaired"
    A->>A: Collect: name, phone, email, address, job type, date/time
    A->>I: book_slot {customerInfo, address, jobType, dateTime}
    I->>API: POST /job/create/
    API-->>I: {UUID}
    I->>API: POST /job/assign/ {UUID, User}
    API-->>I: success
    I->>A: result_success {booking_id}
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking {booking_id}
    I->>API: POST /job/update/ {UUID, Status: Canceled}
    API-->>I: success
    I->>A: result_success
    A->>U: "Your appointment has been cancelled"

    Note over U,API: Existing Client
    A->>I: define_user_phone_number {phone}
    I->>API: GET /job/all/?start_date=today
    API-->>I: Jobs list
    I->>I: Filter by phone, keep future only
    I->>A: Store in persona bookings
```

BookingFlow → ApiFlow Chain

```
flowchart TD
    E["book_slot"] --> GATE{{"Booking<br>enabled?"}}
```

```
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| VALIDATE{{"API secret +<br>phone + name +<br>date + jobType?"}}
    VALIDATE -->|"No"| ERR1["workiz_booking_error"]
    VALIDATE -->|"Yes"| CREATE["POST /job/create/<br>{customer, address, jobType,
<br>dateTime, jobSource}"]
    CREATE --> OK1{{"Success +<br>UUID?"}}
    OK1 -->|"No"| ERR2["workiz_booking_error"]
    OK1 -->|"Yes"| ASSIGN["POST /job/assign/<br>{UUID, primaryUserId}"]
    ASSIGN --> OK2{{"Success?"}}
    OK2 -->|"No"| ERR3["workiz_booking_error"]
    OK2 -->|"Yes"| OUT["workiz_booking_success<br>{booking_id, assigned_user}"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{{"Cancellation<br>enabled?"}}
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| VALIDATE{{"API secret +<br>booking_id?"}}
    VALIDATE -->|"No"| ERR["workiz_cancellation_error"]
    VALIDATE -->|"Yes"| CANCEL["POST /job/update/<br>{UUID, Status: Canceled}"]
    CANCEL --> OK{{"Success?"}}
    OK -->|"No"| ERR2["workiz_cancellation_error"]
    OK -->|"Yes"| OUT["workiz_cancellation_success<br>{booking_id}"]
```

ExistingClientFlow

```
flowchart TD
    E["define_user_phone_number"] --> PHONE{{"Phone<br>available?"}}
    PHONE -->|"No"| SKIP["Return"]
    PHONE -->|"Yes"| API["GET /job/all/<br>?start_date=today"]
    API --> OK{{"Success?"}}
    OK -->|"No"| ERR["workiz_existing_client_search_error"]
    OK -->|"Yes"| FILTER["Filter jobs by phone<br>Keep future dates only"]
    FILTER --> STORE["SetPersonaAttribute<br>field=bookings"]
```

NetworkFlow — Production vs DevMode

```
flowchart TD
    REQ["workiz_execute_request"] --> MODE{{"workiz_mode?"}}
    MODE -->|"Production"| REAL["_executeRequestSkill<br>POST
{base_url}/{api_key}/{endpoint}"]
    MODE -->|"DevMode"| EMULATE["_emulateRequestSkill<br>Gen() → LLM
generates<br>realistic response"]
    REAL --> CONNECTOR["HTTP Connector<br>workiz_connector"]
    CONNECTOR --> RESP{{"statusCode<br>200/201?<br>flag == true?"}}
    RESP -->|"Yes"| SUCCESS["workiz_result_success"]
    RESP -->|"No"| ERROR["workiz_result_error"]
    EMULATE --> SUCCESS
```

4.4 Canvas Scenarios

When `workiz_setup_scenarios` is `True` (default), the CanvasSetupFlow automatically adds:

Scenarios:

- **"Scheduling Appointment via Agent"** (`workiz_schedule_appointment_via_agent_scenario`) — 10-step guided booking flow that collects all required fields (name, phone, email, address, ZIP, job type, date/time) with code-phrases
- **"Canceling Appointment via Agent"** (`cancellation_via_agent_scenario`) — 6-step cancellation flow with confirmation and code-phrases

Intent Types:

- **"[L] Appointment via Agent"** (`library_intent_types_common_appointment_agent`) — from library
- **"[T] Cancellation via Agent"** (`cancellation_via_agent_intent`) — caller wants to cancel

4.5 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice).

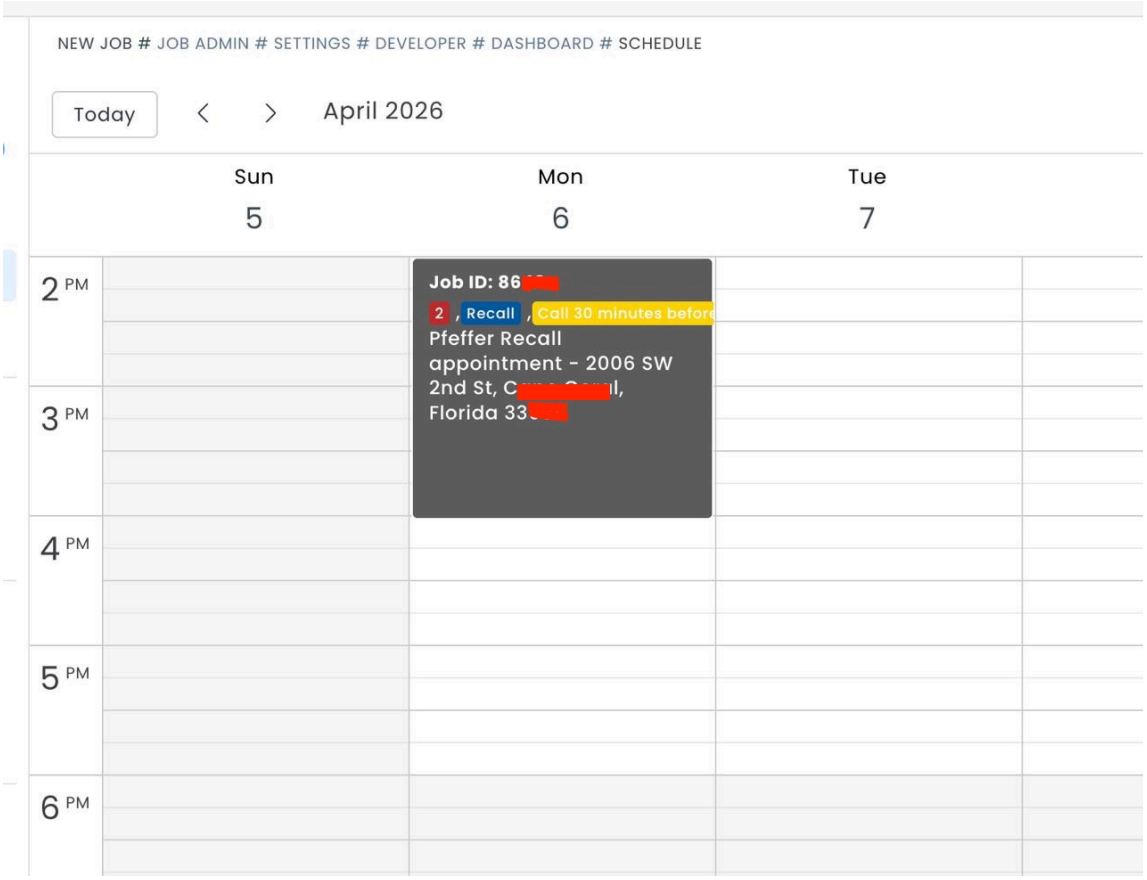
DevMode: Set `workiz_mode` to `DevMode` to test without real API credentials. The integration will use LLM to generate realistic Workiz API responses.

4.6 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify team sync completed (check `workiz_dispatcher_team_members` attribute)
2. **Booking**: Request a service — provide name, phone, email, address, job type, and date → verify job created in Workiz
3. **Cancellation**: Ask to cancel — verify job status changed to "Canceled" in Workiz
4. **Existing Client**: Call with a known phone number → verify agent finds existing bookings

> 📸 *Screenshot: Workiz dashboard showing a job created by the AI agent*



If no action occurs:

- Ensure API credentials are set (`workiz_api_key` , `workiz_api_secret`)
- Verify `workiz_mode` is `Production` (not `DevMode`)
- Check feature flags (`workiz_enable_booking` , `workiz_enable_cancellation`)
- Confirm `workiz_job_types` array matches your Workiz job types exactly
- Verify `workiz_primary_assigned_user_id` is selected (must publish first to populate dropdown)
- Re-publish after configuration changes

> *Note: This integration creates real jobs in Workiz. Jobs and cancellations are reflected in the live Workiz system.*

5. FAQ

Q: How does authentication work? A: Workiz uses two credentials: **API Key** (embedded in the URL path) and **API Secret** (sent in the POST body as `auth_secret`). No OAuth flow needed — just paste the values from Workiz Settings.

Q: What is DevMode? A: When `workiz_mode` is set to `DevMode` , the integration uses LLM (Gen) to emulate Workiz API responses instead of making real HTTP requests. Useful for testing the booking flow without creating real jobs.

Q: How is the technician assigned to a job? A: After creating a job, the integration automatically calls `POST /job/assign/` with the `workiz_primary_assigned_user_id` selected in the platform dropdown. This dropdown is populated from Workiz team members with dispatch/manager/admin roles.

Q: Why doesn't availability checking work? A: Workiz API does not provide an availability/capacity endpoint. The agent collects the preferred date/time from the caller and creates the job at that time. Scheduling conflicts are handled by dispatchers in Workiz.

Q: How does existing client lookup work? A: When `define_user_phone_number` event fires, the integration fetches all jobs from today onward (`GET /job/all/?start_date=today`), filters by the caller's phone number, and stores matching future bookings in the persona's `bookings` attribute. This allows the cancellation flow to find the booking.

Q: What job types are supported? A: Set `workiz_job_types` as a JSON array matching your Workiz configuration, e.g. `["Dishwasher", "Dryer", "Refrigerator", "Washer"]` . The agent will present these as options and validate the selection before creating the job.

Q: What happens if the phone number format is different? A: The integration normalizes phone numbers automatically — strips formatting characters (`+` , `-` , `(` , `)` , spaces, dots), handles 10-digit (prepends +1) and 11-digit (prepends +) US numbers.